

Assignment no. 2:

Practical Assignments on Data Acquisition and Cleaning

Objective: To develop skills in data acquisition from various sources, web scraping, API integration, and data cleaning and preprocessing.

2.1 Data Importing from Various Sources

A CSV (comma-separated values) file is a plain text file that stores tabular data, like a spreadsheet or database, using commas to separate columns and new lines to separate rows

Importing Data from CSV

```
[1]: import pandas as pd
df_csv=pd.read_csv("train.csv")
print(df_csv.head())
```

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp
0	1	0	3			
1	2	1	1			
2	3	1	3			
3	4	1	1			
4	5	0	3			

Parch	Ticket	Fare	Cabin	Embarked	
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S

```
[2]: print(df_csv.tail())
```

PassengerId	Survived	Pclass	Name	
886	887	0	2	Montvila, Rev. Juozas
887	888	1	1	Graham, Miss. Margaret Edith
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"
889	890	1	1	Behr, Mr. Karl Howell
890	891	0	3	Dooley, Mr. Patrick

Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
886	male	27.0	0	0	211536	13.00	NaN	S
887	female	19.0	0	0	112053	30.00	B42	S
888	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	male	26.0	0	0	111369	30.00	C148	C
890	male	32.0	0	0	370376	7.75	NaN	Q

```
[3]: print(df_csv.loc[0:5])
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	
5	6	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	
5	Moran, Mr. James	male	NaN	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S
5	0	330877	8.4583	NaN	Q

A JSON file is a plain-text file that stores structured data using key-value pairs and arrays

Importing Data from JSON

```
[7]: df_json=pd.read_json("colors.json")
print(df_json.head())
```

	name	hex	rgb
0	Black	#000000	{'r':...
1	White	#FFFFFF	{'r':...
2	Red	#FF0000	{'r':...
3	Green	#00FF00	{'r':...
4	Blue	#0000FF	{'r':...

```
[8]: print(df_json.tail())
```

	name	hex	rgb
30	Beige	#F5F5DC	{'r':...
31	Lavender	#E6E6FA	{'r':...
32	Mint	#98FF98	{'r':...
33	Peach	#FFCBA4	{'r':...
34	Slate	#708090	{'r':...

```
[9]: print(df_json.loc[0:15])
```

	name	hex	rgb
0	Black	#000000	{'r':...
1	White	#FFFFFF	{'r':...
2	Red	#FF0000	{'r':...
3	Green	#00FF00	{'r':...
4	Blue	#0000FF	{'r':...
5	Yellow	#FFFF00	{'r':...
6	Cyan	#00FFFF	{'r':...
7	Magenta	#FF00FF	{'r':...
8	Orange	#FFA500	{'r':...
9	Pink	#FFC0CB	{'r':...
10	Purple	#800080	{'r':...
11	Violet	#EE82EE	{'r':...
12	Brown	#A52A2A	{'r':...
13	Maroon	#800000	{'r':...
14	Olive	#808000	{'r':...
15	Teal	#008080	{'r':...

A **SQL database** is a highly structured data management system that organizes information into tables consisting of rows and columns, much like a sophisticated spreadsheet.

SQLite3 is a self-contained, serverless, zero-configuration relational database engine used to store and manage structured data locally within applications.

Importing Data from SQL Databases

```
[1]: import sqlite3 as sl
import pandas as pd
conn = sl.connect('company.db')
cursor = conn.cursor()
cursor.execute('''
CREATE TABLE IF NOT EXISTS bank_records(
    acc_no INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    acc_type TEXT,
    balance REAL
)
''')
conn.commit()
sample_data=[
    (1,'Alice','Saving',95000),
    (2,'Bob','Buisness',150000),
    (3,'Carla','Recurring',5000),
    (4,'Daniel','Saving',70000)
]
cursor.executemany(
    'INSERT INTO bank_records VALUES(?,?,?,?)',
    sample_data
)
conn.commit()

df=pd.read_sql_query("SELECT * FROM bank_records", conn)

print(df.head())

conn.close()
```

	acc_no	name	acc_type	balance
0	1	Alice	Saving	95000.0
1	2	Bob	Buisness	150000.0
2	3	Carla	Recurring	5000.0
3	4	Daniel	Saving	70000.0

2.2 Data Scraping Techniques

Web Scraping with BeautifulSoup:

BeautifulSoup is a Python library used for web scraping. It helps parse HTML and XML documents making it easy to navigate and extract specific parts of a webpage. This article explains the steps of web scraping using BeautifulSoup.

Identify a website with tabular data

[3]:

```
import pandas as pd
import requests
from bs4 import BeautifulSoup
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'
headers = {'User-Agent': 'Mozilla/5.0'}
response = requests.get(url, headers=headers)
print("Status Code:", response.status_code)
```

Status Code: 200

Use BeautifulSoup to scrape the data and store it in a DataFrame.

[4]:

```
soup = BeautifulSoup(response.content, 'html.parser')
tables = soup.find_all("table")
table = tables[0]
rows = table.find_all("tr")
data = []
```

```
for table in tables:
    rows = table.find_all("tr")
    for row in rows:
        cols = row.find_all(["th", "td"])
        if len(cols) >= 4:
            country = cols[0].get_text(strip=True)
            gdp = cols[1].get_text(strip=True)
            year = cols[2].get_text(strip=True)
            data.append([country, gdp, year])
        if len(data) > 5:
            break
df_gdp = pd.DataFrame(
    data,
    columns=['Country', 'GDP', 'Year']
)
```

[6]:

```
print(df_gdp.head())
```

	Country	GDP	Year
0	Country/Territory	IMF (2026)[1]	World Bank(2025)[6]
1	World	126,295,331	118,350,166
2	United States	32,383,920	30,769,700
3	China[n 1]	20,851,593	19,498,039
4	Germany	5,452,858	5,050,923

API Integration:

API stands for "Application Programming Interface." In simple terms, it's a set of rules and protocols that allow how different software applications can communicate and interact with each other. APIs define the methods and data formats that applications can use to request and exchange information. To retrieve data from a web server, a client application initiates a request, and the server responds with the requested data.

Identify a public API (e.g., OpenWeatherMap, CoinGecko).

[6]:

```
import json
import requests

city_name = input("Enter City name :")
api_key = "070c21d42fd8d0af8d4bbfcd3d487988"

api_url = f"https://api.openweathermap.org/data/2.5/weather?q={city_name}&appid={api_key}&units=m

get_info = requests.get(api_url)
print(get_info)
```

Enter City name : Pune
<Response [200]>

Use Python to fetch data from the API and store it in a DataFrame.

[7]:

```
data = get_info.json()

df = pd.json_normalize(data)

print("\nWeather Data:")
print(df)
```

Weather Data:

```
0 [{"id": 804, 'main': 'Clouds', 'description': ... weather base visibility \
stations 10000

dt timezone id name cod coord.lon coord.lat ... \
0 1787638763 19800 1259229 Pune 200 73.8553 18.5196 ...

main.humidity main.sea_level main.grnd_level wind.speed wind.deg \
0 59 1010 938 6.22 264

wind.gust clouds.all sys.country sys.sunrise sys.sunset
0 8.1 85 IN 1787618921 1787664318

[1 rows x 25 columns]
```

Display the first few rows of the DataFrame.

[8]:

```
print("\nFirst 5 rows:")
print(df.head())
```

First 5 rows:

```
      weather      base visibility \
0  [{'id': 804, 'main': 'Clouds', 'description': ... stations      10000

      dt  timezone      id  name  cod  coord.lon  coord.lat  ... \
0  1787638763      19800  1259229  Pune  200      73.8553      18.5196  ...

      main.humidity  main.sea_level  main.grnd_level  wind.speed  wind.deg \
0              59              1010              938              6.22      264

      wind.gust  clouds.all  sys.country  sys.sunrise  sys.sunset
0          8.1          85          IN  1787618921  1787664318
```

[1 rows x 25 columns]

```
import json
import requests
import pandas as pd

coin_name = input("Enter Coin name : ")

api_key = "CG-JW1ahbN6XWEPvZRSJbcvxb3C"

api_url = f"https://api.coingecko.com/api/v3/simple/price?ids={coin_name}&vs_currencies=usd,inr&i

headers = {
    "x-cg-demo-api-key": api_key
}

get_info = requests.get(api_url, headers=headers)

print(get_info)
```

Enter Coin name : bitcoin
<Response [200]>

[7]:

```
data = get_info.json()

df = pd.json_normalize(data)

print("\nCrypto Data:")
print(df)
```

Crypto Data:

```
      bitcoin.usd  bitcoin.usd_24h_change  bitcoin.inr  bitcoin.inr_24h_change
0          78865             1.534697      7478426             0.902824
```

Handling Missing Values:

Create a DataFrame with some missing values.

[8]:

```
import pandas as pd
import numpy as np
print("This is the Dataset containg missing value")
data = {
    'Name': ['Amit', 'Riya', 'Sanjay', 'Priya', 'Neha', 'Riya'],
    'Age': [25, np.nan, 30, np.nan, 28, np.nan],
    'Salary': [50000, 60000, np.nan, 45000, np.nan, 60000],
    'City': ['Pune', 'Mumbai', 'Delhi', np.nan, 'Pune', 'Mumbai']
}
df=pd.DataFrame(data)
print(df)
```

This is the Dataset containg missing value

	Name	Age	Salary	City
0	Amit	25.0	50000.0	Pune
1	Riya	NaN	60000.0	Mumbai
2	Sanjay	30.0	NaN	Delhi
3	Priya	NaN	45000.0	NaN
4	Neha	28.0	NaN	Pune
5	Riya	NaN	60000.0	Mumbai

Use Python to handle missing values by dropping rows,filling with a specific value, and filling with the mean of the column.

[9]:

```
missing_value_boolean = df.isnull()
print("Boolean DataFrame indicating missing value (using isnull) : \n" , missing_value_boolean)
missing_value_boolean = df.isna()
print("Boolean DataFrame indicating missing value (using is na) : \n" , missing_value_boolean)
```

Boolean DataFrame indicating missing value (using isnull) :

	Name	Age	Salary	City
0	False	False	False	False
1	False	True	False	False
2	False	False	True	False
3	False	True	False	True
4	False	False	True	False
5	False	True	False	False

Boolean DataFrame indicating missing value (using is na) :

	Name	Age	Salary	City
0	False	False	False	False
1	False	True	False	False
2	False	False	True	False
3	False	True	False	True
4	False	False	True	False
5	False	True	False	False

```

missing_value_boolean = df.notnull()
print("Boolean DataFrame indicating missing value (using notnull) : \n" , missing_value_boolean)
missing_value_boolean = df.isna()
print("Boolean DataFrame indicating missing value (using isna) : \n" , missing_value_boolean)

```

Boolean DataFrame indicating missing value (using notnull) :

	Name	Age	Salary	City
0	True	True	True	True
1	True	False	True	True
2	True	True	False	True
3	True	False	True	False
4	True	True	False	True
5	True	False	True	True

Boolean DataFrame indicating missing value (using isna) :

	Name	Age	Salary	City
0	True	True	True	True
1	True	False	True	True
2	True	True	False	True
3	True	False	True	False
4	True	True	False	True
5	True	False	True	True

[11]:

```
print(df.isnull().sum())
```

```

Name      0
Age       3
Salary    2
City      1
dtype: int64

```

```

duplicate_rows = df.duplicated()
print("Boolean DataFrame indicating duplicate rows:\n", duplicate_rows)

```

Boolean DataFrame indicating duplicate rows:

```

0    False
1    False
2    False
3    False
4    False
5     True
dtype: bool

```

[13]:

```
print("Duplicate rows:\n", df[df.duplicated()])
```

Duplicate rows:

	Name	Age	Salary	City
5	Riya	NaN	60000.0	Mumbai

[14]:

```
print("Number of duplicate rows:", df.duplicated().sum())
```

Number of duplicate rows: 1

[15]:

```

df_dropped = df.dropna()
print("After dropping rows with missing values: ")
print(df_dropped)

```

After dropping rows with missing values:

	Name	Age	Salary	City
0	Amit	25.0	50000.0	Pune


```
df1 = df.copy()
print(df1)
```

	Name	Age	Salary	City
0	Amit	25.0	50000.0	Pune
1	Riya	NaN	60000.0	Mumbai
2	Sanjay	30.0	NaN	Delhi
3	Priya	NaN	45000.0	NaN
4	Neha	28.0	NaN	Pune
5	Riya	NaN	60000.0	Mumbai

[17]:

```
df.fillna('Unknown', inplace=True)
print(df)
```

	Name	Age	Salary	City
0	Amit	25.0	50000.0	Pune
1	Riya	Unknown	60000.0	Mumbai
2	Sanjay	30.0	Unknown	Delhi
3	Priya	Unknown	45000.0	Unknown
4	Neha	28.0	Unknown	Pune
5	Riya	Unknown	60000.0	Mumbai

```
print("Replacing missing value in 'Age' by the mean of the row: \n")
df1['Age'].fillna(df1['Age'].mean(), inplace=True)
print(df1)
```

Replacing missing value in 'Age' by the mean of the row:

	Name	Age	Salary	City
0	Amit	25.000000	50000.0	Pune
1	Riya	27.666667	60000.0	Mumbai
2	Sanjay	30.000000	NaN	Delhi
3	Priya	27.666667	45000.0	NaN
4	Neha	28.000000	NaN	Pune
5	Riya	27.666667	60000.0	Mumbai

[19]:

```
print("Replacing missing value in 'Salary' by the median of the row: \n")
df1['Salary'].fillna(df1['Salary'].median(), inplace=True)
print(df1)
```

Replacing missing value in 'Salary' by the median of the row:

	Name	Age	Salary	City
0	Amit	25.000000	50000.0	Pune
1	Riya	27.666667	60000.0	Mumbai
2	Sanjay	30.000000	55000.0	Delhi
3	Priya	27.666667	45000.0	NaN
4	Neha	28.000000	55000.0	Pune
5	Riya	27.666667	60000.0	Mumbai

Handling Outliers:

Create a DataFrame with some outliers.

[55]:

```
import pandas as pd
import numpy as np
from scipy.stats import zscore
print("Original Dataset:")
data = {
    'Name': ['Amit', 'Riya', 'Sanjay', 'Priya', 'Neha',
            'Raj', 'Karan', 'Suresh', 'Anita', 'Rahul',
            'Pooja', 'Vikas', 'Meena', 'Arjun', 'Sneha'],
    'Marks': [78, 82, 85, 80, 88, 76, 84, 81, 87, 79,
            83, 86, 80, 82, 0]
}
df=pd.DataFrame(data)
print(df)
```

Original Dataset:

	Name	Marks
0	Amit	78
1	Riya	82
2	Sanjay	85
3	Priya	80
4	Neha	88
5	Raj	76
6	Karan	84
7	Suresh	81
8	Anita	87
9	Rahul	79
10	Pooja	83
11	Vikas	86
12	Meena	80
13	Arjun	82
14	Sneha	0

[56]:

```
df['Z_Score'] = zscore(df['Marks'])
threshold = 3
outliers = df[np.abs(df['Z_Score']) > threshold]
print("\nOutliers:")
print(outliers)
```

Outliers:

	Name	Marks	Z_Score
14	Sneha	0	-3.694991

Remove the outliers and display the cleaned DataFrame.

[58]:

```
cleaned_df = df[np.abs(df['Z_Score']) <= threshold]
cleaned_df = cleaned_df.drop(columns=['Z_Score'])
print("\nCleaned DataFrame:")
print(cleaned_df)
```

Cleaned DataFrame:

	Name	Marks
0	Amit	78
1	Riya	82
2	Sanjay	85
3	Priya	80
4	Neha	88
5	Raj	76
6	Karan	84
7	Suresh	81
8	Anita	87
9	Rahul	79
10	Pooja	83
11	Vikas	86
12	Meena	80
13	Anjun	82